
AAA Formalización de la lógica modal K en Lean: sintáxis, semántica de Kripke y adecuación



UNIVERSIDAD
COMPLUTENSE
MADRID

Jorge Martinena Cepa
Facultad de Ciencias Matemáticas
Universidad Complutense de Madrid

Trabajo de Fin de Grado
Grado en Matemáticas
(Especialización en Ciencias de la Computación)

Septiembre de 2026

Dirigido por Ignacio Fábregas y Óscar Martín

Resumen

Este trabajo presenta la formalización en el asistente de demostración Lean 4 de la lógica modal $\mathbf{K}$ y de sus dos resultados fundamentales: los teoremas de corrección y completitud respecto a la semántica de Kripke. Frente al estilo conciso de una biblioteca como Mathlib, esta formalización se ha desarrollado con una intención deliberadamente didáctica para hacer visible cada paso del razonamiento lógico. La implementación abarca desde la sintaxis básica hasta la construcción del modelo canónico y una versión del lema de Lindenbaum basada en el lema de Zorn. La memoria expone los fundamentos lógicos y computacionales necesarios sin presuponer conocimientos previos, justificando las decisiones de diseño y las tácticas empleadas, y culmina con un ejemplo ilustrativo de aplicación AAA.

Tabla de contenidos

Resumen	I
1. Introducción	1
1.1. Objetivos	2
1.2. Metodología y plan de trabajo	3
1.3. Estructura de la memoria	4
2. El asistente de demostración Lean	5
2.1. La demostración asistida por ordenador	5
2.2. El fundamento: la teoría de tipos	6
2.2.1. Todo término tiene un tipo	6
2.2.2. Las proposiciones son tipos	7
2.2.3. Tipos inductivos	8
2.3. Definiciones y enunciados	9
2.4. El modo táctico y el InfoView	10
2.5. Catálogo de tácticas empleadas	12
2.5.1. Tácticas básicas de manipulación de objetivos	12
2.5.2. Construir y descomponer estructuras lógicas	13
2.5.3. Razonamiento clásico	13
2.5.4. Inducción y reescritura	14
2.6. Mathlib, la lógica clásica y el lema de Zorn	15
3. La lógica modal $\mathbf{K}$	17
3.1. ¿Necesidad y posibilidad?	17
3.2. Sintaxis	18
3.3. El sistema axiomático $\mathbf{K}$	19
3.4. Semántica de Kripke	21
3.5. Corrección y completitud: el mapa de la demostración	23

4. La formalización en Lean	26
4.1. Visión general y decisiones de diseño	26
4.2. La sintaxis	26
4.3. El esquema de tautologías	26
4.4. El sistema axiomático	26
4.5. La semántica de Kripke	26
4.6. El teorema de corrección	26
4.7. El teorema de completitud	27
4.7.1. Deducción a partir de hipótesis	27
4.7.2. Consistencia y conjuntos maximales consistentes	27
4.7.3. El lema de Lindenbaum, vía Zorn	27
4.7.4. El modelo canónico y el lema de encajonado	27
4.7.5. Los lemas de existencia y de verdad	27
4.7.6. Completitud y adecuación	27
5. Hacia la lógica epistémica: el problema de los niños embarrados	28
6. Conclusiones	29
Bibliografía	30

Capítulo 1

Introducción

Sobre Lean

Una demostración matemática es, en principio, un objeto completamente preciso: una cadena de pasos en la que cada afirmación se sigue de las anteriores mediante reglas aceptadas. En la práctica, sin embargo, las demostraciones que escribimos y leemos están llenas de atajos: pasos “evidentes” que se omiten, casos “análogos” que no se comprueban o notación que se sobreentiende. En general, este nivel de detalle es el adecuado para hacer matemáticas entre personas, pero deja abierta una pregunta incómoda respecto de la formalización pura: ¿cómo sabemos que no se nos ha escapado nada?

Los asistentes de demostración atacan esta pregunta de frente. Son programas que permiten escribir definiciones, enunciados y demostraciones en un lenguaje formal, y que solo aceptan una demostración cuando cada uno de sus pasos ha sido verificado mecánicamente a partir de los axiomas del sistema y los resultados previamente introducidos y comprobados. La idea no es nueva. Se remonta a proyectos como Automath (1967) o Mizar (1973). Además, en los últimos años ha ganado un impulso enorme: sistemas como Rocq o Lean cuentan hoy con comunidades muy activas y con bibliotecas que cubren buena parte del grado en Matemáticas y más allá [7]. En este trabajo se usa Lean 4, por la cantidad de material abierto para su aprendizaje en línea y la cultura de uso que está comenzando a generarse en nuestra Facultad.

En lo personal, los asistentes de demostración, con sus respectivas bibliotecas, despiertan un interés particular por el acercamiento aparente que presentan al ideal de construcción de un "árbol de las matemáticas". Esto es, la posibilidad de tener, de forma tangible, un grafo de todos los teoremas y resultados matemáticos y cómo se derivan unos de otros. A grandes rasgos y de forma ingenua, esto tendría dos implicaciones muy interesantes en direcciones opuestas: en un sentido descendente, poder generar un árbol de dependencias para cada resultado, teorema, etc. En otro ascendente, la posibilidad de que, el avance tecnológico y de la inteligencia artificial, unidos a la posibilidad de verificación mecánica de resultados, permitan avanzar en un futuro en la demostración de problemas abiertos. Este ideal es, como se ha mencionado antes, atractivo pero ingenuo por varias

razones, algunas de las cuales se enfrentarán en el desarrollo de este trabajo y se comentarán en sus conclusiones.

Sobre lógica modal

Como caso de estudio para aprender a usar un asistente de demostración se ha elegido la lógica modal. La lógica modal extiende la lógica proposicional clásica con dos operadores nuevos: $\Box$ y $\Diamond$ que, habitualmente, se leen como “*es necesario*” y “*es posible*” respectivamente. Más adelante veremos que esto no es más que su interpretación clásica, pero existen otras dependiendo de la modalidad que se quiera considerar. Su semántica habitual se define sobre los modelos de Kripke, que son esencialmente grafos dirigidos cuyos nodos, llamados *mundos posibles*, llevan asociada una valoración de las variables proposicionales. El sistema modal más básico es la lógica **K**, que se obtiene añadiendo al cálculo proposicional el axioma K, que distribuye $\Box$ sobre la implicación, y la regla de necesidad, que permite concluir $\Box\varphi$ de φ . Los dos resultados centrales de la teoría son los teoremas de *corrección* y *completitud*: todo lo demostrable en **K** es válido en todos los modelos de Kripke y, recíprocamente, todo lo válido es demostrable.

La lógica modal resulta un caso de estudio para este trabajo especialmente apropiado por varios motivos. Primero, es un tema autocontenido: su sintaxis y su cálculo se definen desde cero, sin apoyarse en grandes construcciones previas, lo que permite formalizarlo casi entero en lugar de importar la mitad del trabajo de una biblioteca. Segundo, tiene una dificultad suficiente: la corrección es un ejercicio asequible de inducción, mientras que la completitud, con sus múltiples pasos intermedios, es una demostración no trivial que pone a prueba tanto al asistente como a quien lo maneja. Y tercero, es un tema con recorrido y relevancia en mi rama de especialización: sobre la lógica **K** se construyen las lógicas epistémicas y temporales que se usan en informática para razonar sobre conocimiento y sobre programas.

Hay que señalar que la lógica modal ya tiene formalizaciones con un estilo más sintético, similar al de la biblioteca Mathlib, y con importante carga de importaciones de la misma (AAA referencias). Este trabajo no pretende competir con esas formalizaciones, sino hacer algo distinto con ellas como referencia lejana: una formalización didáctica, pensada para ser leída. Las demostraciones de Mathlib están optimizadas para ser mantenibles y compactas, y con frecuencia resultan casi ilegibles para quien no es experto. Las expuestas en este trabajo, en cambio, están escritas paso a paso, comentadas, y eligiendo en cada momento la táctica que mejor refleja el razonamiento matemático que se está haciendo, aunque no sea necesariamente lo más óptimo para un asistente de demostración.

1.1. Objetivos

El objetivo general del trabajo es formalizar en Lean 4 la lógica modal **K** y demostrar en este entorno su corrección y completitud respecto de los modelos de Kripke, tal como fija la propuesta

del TFG. Este objetivo general se concreta en los siguientes objetivos específicos:

1. Aprender el manejo de Lean 4 como asistente de demostración partiendo de cero: su fundamento en teoría de tipos, su lenguaje de definiciones y su modo táctico.
2. Estudiar la lógica modal $\mathbf{K}$ y su semántica de Kripke, siguiendo principalmente el texto de Blackburn, de Rijke y Venema [3], hasta la demostración clásica de completitud por modelos canónicos.
3. Formalizar la sintaxis, el sistema axiomático y la semántica, y demostrar formalmente los teoremas de corrección y completitud, junto con los resultados auxiliares que requieren (teorema de deducción, lema de Lindenbaum, lema de verdad).
4. AAA Como ampliación, explorar una aplicación de la maquinaria anterior al razonamiento epistémico, tomando como hilo conductor el problema de los niños embarrados (*muddy children*).

1.2. Metodología y plan de trabajo

El trabajo se ha desarrollado en tres fases que, en la práctica, se han solapado constantemente. Debe tenerse en cuenta que, para el presente trabajo se ha tenido que aprender un software desconocido para formalizar un contenido matemático igualmente desconocido.

Fase de aprendizaje. El primer contacto con Lean se realizó a través de materiales introductorios de la comunidad: el *Natural Number Game* [4], el libro *Theorem Proving in Lean 4* [1] y *Mathematics in Lean* [2]. En paralelo se estudió la parte de lógica modal, principalmente los capítulos 1 y 4 de [3], que cubren sintaxis, semántica y la completitud por modelos canónicos.

Fase de formalización. La formalización se construyó de forma incremental, en el mismo orden en que se presenta en el capítulo 4: sintaxis, sistema axiomático, semántica, corrección y, por último, completitud. Cada bloque se validó con ejemplos y algunos lemas que sirvieran de verificación y ejemplificación (por ejemplo, comprobar que las conectivas derivadas reciben el significado semántico esperado) antes de pasar al siguiente. Cabe destacar que hay dos decisiones de diseño separan nuestra formalización del desarrollo en papel de [3] y se justifican con detalle en su momento: la noción de deducibilidad desde hipótesis se define de manera inductiva (en lugar de mediante conjunciones finitas de premisas) y el lema de Lindenbaum se demuestra importando el lema de Zorn (eliminando la hipótesis de numerabilidad del lenguaje).

Fase de redacción. La memoria se ha escrito con el código ya estable, lo que ha permitido que cada táctica y cada construcción de Lean que se explica en el capítulo 2 aparezca realmente usada en el capítulo 4, y recíprocamente. Es decir, no se explica nada que no se use, ni se usa nada que no se explique.

AAA Nota sobre el uso de IA

Conviene hacer una aclaración sobre las herramientas empleadas. La inmensa mayoría de las demostraciones se han escrito a mano, con la ayuda del *InfoView* de Lean. En un número reducido de teoremas técnicos se ha recurrido a herramientas de inteligencia artificial, en concreto Claude, como apoyo para encontrar las tácticas adecuadas para el desarrollo de la demostración. En todos los casos, las demostraciones han sido escritas y desarrolladas manualmente, revisadas, comentadas línea a línea y, por supuesto, verificadas por Lean, que es en última instancia quien garantiza su corrección.

1.3. Estructura de la memoria

El resto de la memoria se organiza como sigue.

El capítulo 2 presenta Lean: qué es un asistente de demostración, en qué teoría de tipos se fundamenta, cómo se escriben definiciones y demostraciones, y un catálogo razonado de todas las tácticas que se emplean en este trabajo.

El capítulo 3 desarrolla la parte matemática: la sintaxis del lenguaje modal, el sistema **K**, la semántica de Kripke y el mapa de la demostración de completitud que después se formaliza.

El capítulo 4 es el núcleo del trabajo: recorre la formalización completa, desde el tipo inductivo de las fórmulas hasta el teorema de adecuación, explicando y justificando cada decisión de diseño y cada demostración.

AAAEl capítulo 5 esboza la ampliación hacia la lógica epistémica y el problema de los niños embarrados.

Finalmente, el capítulo 6 recoge las conclusiones y otros comentarios finales.

Capítulo 2

El asistente de demostración Lean

Este capítulo presenta Lean con un criterio concreto: explicar lo aprendido en el manejo de la herramienta y lo necesario para la formalizar el capítulo 4, cosas equivalentes. Empezamos situando Lean entre los asistentes de demostración (sección 2.1); después describimos su fundamento teórico, la teoría de tipos dependientes, en la medida en que hace falta para entender qué significa que “una demostración es un término” (sección 2.2); a continuación explicamos cómo se escriben definiciones y teoremas (sección 2.3); cómo funciona el modo táctico (sección 2.4); y cerramos con un catálogo razonado de todas las tácticas que se usan en este trabajo (sección 2.5) y con las herramientas de Mathlib de las que dependemos, en particular la lógica clásica y el lema de Zorn (sección 2.6).

2.1. La demostración asistida por ordenador

Un asistente de demostración es un programa con dos componentes bien diferenciadas. Por un lado, un lenguaje formal en el que el usuario escribe definiciones, enunciados y demostraciones. Por otro, un núcleo (*kernel*): un verificador pequeño que comprueba, paso a paso, que cada demostración es correcta según las reglas de la lógica subyacente. Todo lo demás (editor, tácticas, automatización...) son capas de comodidad: pueden ayudar a construir la demostración, pero la única autoridad que la acepta es el núcleo. Esta arquitectura, conocida como criterio de De Bruijn (AAA), es lo que hace razonable fiarse del sistema: para dudar de un teorema verificado hay que dudar de un programa de unos pocos miles de líneas, no de los cientos de miles que forman el resto de la herramienta.

Los primeros sistemas de este tipo, como Automath (1967) y Mizar (1973), demostraron que la idea era viable. Hoy los asistentes más usados en matemáticas son Rocq, Isabelle/HOL y Lean. Lean [5], desarrollado inicialmente por Leonardo de Moura en Microsoft Research y mantenido actualmente por la Lean Focused Research Organisation, es a la vez un asistente de demostración y un lenguaje de programación funcional. En este trabajo usamos su cuarta versión, Lean 4, junto con su biblioteca matemática Mathlib [7], un proyecto comunitario que reúne, verificadas, buena parte de las matemáticas de grado y posgrado. De Mathlib no importaremos resultados de lógica

modal (el objetivo es construirla nosotros), pero sí su infraestructura básica: la teoría elemental de conjuntos, el modo táctico y, en un punto crucial, el lema de Zorn.

En la práctica, trabajar con Lean se parece a mantener un diálogo. El usuario escribe en el editor y una ventana lateral, el *InfoView*, muestra en cada momento el estado de la demostración: qué hipótesis hay disponibles y qué falta por demostrar. Cada instrucción del usuario transforma ese estado, y la demostración termina cuando no queda nada pendiente por probar. Veremos ese mecanismo con detalle en la sección 2.4.

2.2. El fundamento: la teoría de tipos

La lógica sobre la que se construye Lean no es la teoría de conjuntos de Zermelo–Fraenkel, sino una *teoría de tipos dependientes* llamada cálculo de construcciones con tipos inductivos. No necesitamos conocer este cálculo en profundidad, pero sí entender tres de sus ideas, porque aparecen constantemente en la formalización: 1) todo término tiene un tipo, 2) las proposiciones son tipos, y 3) los tipos inductivos.

2.2.1. Todo término tiene un tipo

En teoría de tipos, los objetos básicos son los términos, y cada término tiene asignado un tipo. La notación $x : T$ se lee “ x es un término de tipo T ” y es la afirmación fundamental del sistema; juega un papel análogo al de la pertenencia $x \in A$ en teoría de conjuntos, con una diferencia importante: un término tiene un único tipo, que puede calcularse mecánicamente. Por ejemplo, $(2 : \mathbf{Nat})$ declara que 2 es un número natural, y $\mathbf{Nat}$ es a su vez un término cuyo tipo es $\mathbf{Type}$, el “tipo de los tipos” de datos.¹

Los tipos de funciones se escriben con una flecha: una función de los naturales en los naturales tiene tipo $\mathbf{Nat} \rightarrow \mathbf{Nat}$, y si $f : \mathbf{Nat} \rightarrow \mathbf{Nat}$ y $n : \mathbf{Nat}$, la aplicación se escribe por yuxtaposición, $f\ n$, sin paréntesis. Las funciones de varios argumentos se representan encadenando flechas, que asocian a la derecha: $\mathbf{Nat} \rightarrow \mathbf{Nat} \rightarrow \mathbf{Nat}$ es el tipo de las funciones que reciben un natural y devuelven una función $\mathbf{Nat} \rightarrow \mathbf{Nat}$; en la práctica, una función de dos argumentos. Las funciones anónimas se escriben con la palabra clave `fun`: por ejemplo, `fun n => n + 1` es la función que a cada n le asigna $n + 1$. Esta notación aparecerá al definir modelos concretos, como `R := fun _ _ => False` (la relación que no relaciona nada con nada; el guion bajo indica un argumento cuyo nombre no interesa).

La teoría es dependiente porque el tipo del resultado de una función puede depender del valor del argumento. El ejemplo que nos importa es el cuantificador universal: el tipo $\forall x : A, P\ x$ es el tipo de las funciones que a cada $x : A$ le asignan un término de tipo $P\ x$, donde $P\ x$ varía con x . Esta lectura del $\forall$ como un tipo de funciones tiene consecuencias muy prácticas que veremos enseguida.

¹Para evitar paradojas del estilo de la de Russell, $\mathbf{Type}$ no puede ser término de sí mismo: Lean organiza los tipos en una jerarquía infinita $\mathbf{Type}\ 0, \mathbf{Type}\ 1, \mathbf{Type}\ 2, \dots$ llamada *jerarquía de universos*. En este trabajo no hará falta subir nunca de $\mathbf{Type}\ 0$ (que abrevia $\mathbf{Type}\ 0$) y $\mathbf{Prop}$, así que no volveremos a mencionar los universos.

2.2.2. Las proposiciones son tipos

Junto a `Type`, Lean tiene otro universo llamado `Prop`: el tipo de las proposiciones. Un término $p : \text{Prop}$ es un enunciado (“2 es par”, “ φ es válida”), y la idea central del sistema, conocida como correspondencia de Curry–Howard o *proposiciones como tipos*, es la siguiente:

Una demostración de la proposición p es un término de tipo p .

Es decir, las proposiciones se tratan exactamente igual que los demás tipos, y demostrar p consiste en construir un término $h : p$. Si no existe ningún término de tipo p , la proposición no es demostrable. Bajo esta correspondencia, las conectivas lógicas se convierten en constructores de tipos y su comportamiento queda determinado por el de los tipos correspondientes:

- **Las que son funciones.** Su demostración es un procedimiento que transforma demostraciones en demostraciones, y usarla consiste en aplicarlo.
 - La implicación $p \rightarrow q$ es el tipo de las funciones de p en q : demostrarla es dar una función que convierte demostraciones de p en demostraciones de q . Por eso, si $h : p \rightarrow q$ y $hp : p$, entonces $h\ hp : q$. El modus ponens es la aplicación de funciones.
 - El cuantificador $\forall x : A, P\ x$ es el tipo de funciones dependientes descrito antes: demostrarlo es dar un procedimiento que a cada x le asigna una demostración de $P\ x$, y usarlo sobre un elemento concreto a es aplicar la función: $h\ a : P\ a$.
 - La negación $\neg p$ se define como $p \rightarrow \text{False}$, donde `False` es la proposición sin demostraciones (un tipo vacío) y `True` la que tiene una demostración trivial. Demostrar que p es falsa es, pues, dar una función que convierta cualquier hipotética demostración de p en una demostración de lo imposible. Este detalle será importante: nuestra negación modal $\sim\varphi := \varphi \rightarrow \perp$ imita exactamente esta definición, y esa imitación hará que varios lemas semánticos sean ciertos “por definición”.
- **Las que son pares (o sumas).** Su demostración se construye empaquetando piezas, y usarla consiste en extraerlas.
 - La conjunción $p \wedge q$ es el tipo de los pares: una demostración suya es un par $\langle hp, hq \rangle$ formado por una demostración de cada componente, que se recuperan con `.1` y `.2`.
 - El bicondicional $p \leftrightarrow q$ es un par de implicaciones, cuyas componentes se extraen con `.mp` (de izquierda a derecha) y `.mpr` (de derecha a izquierda).
 - El existencial $\exists x : A, P\ x$ se demuestra con un par $\langle a, ha \rangle$: un testigo a y una demostración $ha : P\ a$. Es a la conjunción lo que el cuantificador universal es a la implicación: un par en el que el tipo de la segunda componente depende de la primera.

- La disyunción $p \vee q$ es la excepción de este grupo: no es un par sino una suma, con dos constructores, `Or.inl hp` y `Or.inr hq`, según qué componente se demuestre. Construir la exige elegir un lado. Usarla, en cambio, obliga a tratar los dos casos posibles.

Los corchetes angulares $\langle \ , \ \rangle$ que acaban de aparecer son el constructor anónimo: una notación uniforme para construir términos de cualquier tipo con un solo constructor. Los tres primeros casos del segundo grupo son instancias de lo mismo, un par, con una o dos componentes que pueden depender entre sí, y por eso los tres se escriben igual; la disyunción, con sus dos constructores, queda fuera. Esta notación aparecerá constantemente, también para los elementos de las estructuras que definamos.

2.2.3. Tipos inductivos

La tercera idea es el mecanismo con el que se crean tipos nuevos: los tipos inductivos. Un tipo inductivo se define dando una lista de constructores, y sus términos son exactamente los que se obtienen aplicando constructores un número finito de veces. El ejemplo clásico son los números naturales, definidos en la biblioteca de Lean esencialmente así:

```
1 inductive Nat where
2   | zero : Nat          -- el cero es un natural
3   | succ : Nat → Nat    -- el sucesor de un natural es un natural
```

Cada línea que empieza por `|` declara un constructor con su tipo. La definición debe leerse como: los términos de tipo `Nat` son `Nat.zero`, `Nat.succ Nat.zero`, `Nat.succ (Nat.succ Nat.zero)`, etcétera, y nada más. De este “y nada más” Lean extrae automáticamente dos herramientas:

- Un *principio de recursión*: para definir una función sobre un tipo inductivo basta decir qué hace con cada constructor. Lo usaremos, por ejemplo, para definir la relación de satisfacción por recursión sobre la estructura de las fórmulas.
- Un *principio de inducción*: para demostrar que todos los términos del tipo cumplen una propiedad basta demostrarla para cada constructor, suponiéndola para sus argumentos (hipótesis de inducción). La inducción usual sobre $\mathbb{N}$ es el caso particular para `Nat`; nosotros usaremos, además, la inducción estructural sobre fórmulas.

Hay un uso de los tipos inductivos menos evidente y absolutamente central en este trabajo: se pueden definir proposiciones inductivas, es decir, familias de términos de `Prop` generadas por constructores. Nuestra noción de demostrabilidad ($\vdash \varphi$) será una proposición inductiva cuyos constructores son, literalmente, los axiomas y reglas del sistema **K**: una demostración en **K** queda representada como un árbol finito de aplicaciones de constructores. La consecuencia es muy potente: el principio

de inducción asociado permite hacer inducción sobre las demostraciones, que es exactamente la técnica con la que se prueba el teorema de corrección (“si los axiomas son válidos y las reglas preservan la validez, todo teorema es válido”).

Por último, las estructuras (**structure**) son el caso particular de tipo inductivo con un único constructor: sirven para empaquetar varios campos en un solo objeto, como un marco de Kripke (un tipo de mundos y una relación) o un modelo (un marco y una valoración). Si **F** es una estructura con un campo **R**, se accede a él con la notación **F.R**; y una estructura puede extender a otra (**extends**), heredando sus campos, como hará **KripkeModel** con **KripkeFrame**. Esto no es de especial relevancia, y se construirá con ánimo meramente ilustrativo.

2.3. Definiciones y enunciados

Con el fundamento anterior, escribir Lean consiste en declarar términos y sus tipos. Las declaraciones que usamos son cuatro:

- **def** introduce una definición con nombre. Por ejemplo, **def EsPar (n : Nat) : Prop := ∃ k : Nat, n = 2 * k** define la propiedad de ser par: recibe un natural **n** y devuelve la proposición que afirma que **n** es el doble de algún **k**. Entre el nombre y los dos puntos van los argumentos con su tipo; lo que sigue a los dos puntos es el tipo del resultado (aquí **Prop**, porque lo definido es un enunciado y no un número), y lo que sigue a **:=** es el término definido. Así se definirán propiedades en el capítulo 4 con nociones como la validez o la consistencia.
- **theorem** declara un teorema: su “tipo” es el enunciado (una proposición) y su “definición” es la demostración (un término de ese tipo). Que teoremas y definiciones tengan la misma forma es un ejemplo más de correspondencia de Curry–Howard.
- **example** es un teorema sin nombre: se verifica y se olvida. Lo usamos para ejemplos o verificaciones que no se necesitan después.
- **variable** declara parámetros comunes a varias declaraciones. En nuestro código, por ejemplo, **variable {VP : Type}** fija de una vez el tipo de las variables proposicionales para todo el desarrollo.

Argumentos implícitos

En algunas declaraciones aparecen argumentos entre llaves en lugar de entre paréntesis. Las llaves indican que el argumento es implícito: quien usa la definición no lo escribe, y Lean lo infiere del contexto. Por ejemplo, un lema sobre listas que afirme que invertir dos veces devuelve la lista de partida, **theorem rev_rev {α : Type} (l : List α) : (l.reverse).reverse = l**, tiene el tipo α implícito: al aplicarlo a una lista concreta de naturales, Lean deduce por sí solo que α es **Nat** y

escribirlo sería redundante. En el código, la regla práctica que se ha seguido es marcar como implícito todo argumento que pueda deducirse del tipo de los demás, y dejar explícitos los que no. En el capítulo 4, el tipo de variables proposicionales sobre el que se construye todo el desarrollo será un argumento implícito de casi todas las definiciones y teoremas.

Notación

Lean permite introducir notación infija y prefija con niveles de precedencia. Por ejemplo,

```
1 local infixr:60 " → " => ModalFormula.imp
2 local prefix:70 "□"   => ModalFormula.box
```

declara que $\varphi \rightarrow \psi$ abrevia `ModalFormula.imp  $\varphi$   $\psi$` y que $\Box\varphi$ abrevia `ModalFormula.box  $\varphi$` . El número tras los dos puntos es la precedencia. A mayor número, más fuerte liga: así $\Box\varphi \rightarrow \psi$ se lee $(\Box\varphi) \rightarrow \psi$. La `r` de `infixr` indica asociatividad a la derecha (la convención habitual para la implicación) y el modificador `local` restringe la notación al fichero actual. Como se discute en la sección 4.2, elegimos deliberadamente símbolos distintos de los de Lean ($\rightarrow$ frente a $\rightarrow$, $\sim$ frente a $\neg$, $\wedge$ frente a $\wedge \dots$) para que siempre sea visible si una expresión es una fórmula modal o una proposición de Lean.

Definiciones por recursión

Como anticipamos, una función sobre un tipo inductivo puede definirse por casos sobre los constructores, mediante ajuste de patrones (*pattern matching*):

```
1 def suma : Nat → Nat → Nat
2   | Nat.zero,   n => n
3   | Nat.succ m, n => Nat.succ (suma m n)
```

Cada línea `| patrón => valor` cubre un constructor; Lean comprueba que los casos son exhaustivos y que las llamadas recursivas se hacen sobre argumentos estrictamente menores, lo que garantiza que la definición termina. Sobre estas definiciones se apoya la noción de *igualdad definicional*: dos expresiones son definicionalmente iguales si una se obtiene de la otra desplegando definiciones y computando. Por ejemplo, `suma Nat.zero n` es definicionalmente `n`, sin más que aplicar la primera cláusula. Las igualdades definicionales no necesitan demostración: la táctica `rf1` (sección 2.5) las cierra de inmediato, y en la formalización señalaremos varios lemas cuya demostración completa es, precisamente, `rf1`. La relación de satisfacción del capítulo 4 se define exactamente así, con una cláusula por cada conectiva del lenguaje modal.

2.4. El modo táctico y el InfoView

En principio, demostrar un teorema es escribir un término de su tipo, y para enunciados sencillos puede hacerse directamente. Por ejemplo, el modus ponens:

```
1 theorem mp {p q : Prop} (h : p → q) (hp : p) : q := h hp
```

La demostración es la aplicación `h hp`, tal como vimos en la sección 2.2: la implicación es una función y aplicarla es razonar. Pero construir a mano el término de una demostración larga es impracticable. Para eso existe el *modo táctico*: escribiendo `by` tras el enunciado, en lugar de un término se da una sucesión de tácticas, instrucciones que van construyendo el término por nosotros.

El modo táctico se entiende mejor con la noción de estado de la demostración, que el InfoView muestra en todo momento. Un estado consta de uno o varios objetivos (*goals*); cada objetivo tiene un contexto de hipótesis con nombre (`n : Nat, h : p, ...`) y una meta, la proposición que falta demostrar, señalada con el símbolo $\vdash$. Cada táctica transforma el estado: puede simplificar la meta, añadir hipótesis, dividir el objetivo en varios o cerrarlo. La demostración acaba cuando no quedan objetivos y el InfoView muestra el mensaje “*Goals accomplished!*”.

Veámoslo con un ejemplo elemental: si p es cierta, entonces p se sigue de cualquier otra cosa. En los comentarios se indica el estado de la demostración antes y después de cada táctica, tal como lo mostraría el InfoView.

```
1 example (p q : Prop) : p → (q → p) := by
2   -- ⊢ p → q → p
3   intro hp
4   -- hp : p
5   -- ⊢ q → p
6   intro hq
7   -- hp : p, hq : q
8   -- ⊢ p
9   exact hp
10  -- Goals accomplished!
```

La meta inicial es una implicación anidada. La táctica `intro hp` supone el antecedente —es el “supongamos p ” de una demostración informal— y lo añade al contexto con el nombre `hp`, dejando como meta el consecuente; `intro hq` hace lo propio con la segunda implicación. La meta es entonces p , que es exactamente la hipótesis `hp`, y `exact hp` la señala y cierra el objetivo. Estas dos tácticas, junto con el resto de las empleadas en el trabajo, se detallan en la sección 2.5.

Merece la pena subrayar la relación entre los dos modos: las tácticas no son una lógica aparte, sino un lenguaje de órdenes para construir el mismo término que podríamos escribir a mano. En el ejemplo, el término construido es `fun hp hq => hp`. A lo largo del trabajo combinamos ambos estilos según cuál resulte más legible —alguna demostración corta se escribe directamente como término, igual que el *modus ponens* de arriba—, con preferencia por el modo táctico con las hipótesis intermedias explícitas (`have`), que es el que más se parece a una demostración escrita en prosa.

2.5. Catálogo de tácticas empleadas

Esta sección recoge todas las tácticas que aparecen en la formalización, agrupadas por función, con su significado matemático y una referencia a algún lugar del capítulo 4 donde se utilicen. La tabla 2.1 sirve de resumen y de índice.

Táctica	Lectura matemática	Uso destacado
intro	sea... / supongamos...	en casi todos los lemas
exact	que es exactamente...	en casi todos los lemas
apply	basta ver... (razonar hacia atrás)	lema de verdad
have	afirmación intermedia	regla RE (§4.4)
rfl	por definición	satisfies_neg (§4.5)
constructor	separar $\leftrightarrow$, $\wedge$ en partes	satisfies_dia
rintro, rcases, obtain	descomponer hipótesis	Lindenbaum (§4.7)
refine	esquema con huecos	lema de existencia
by_contra	reducción al absurdo	completitud
by_cases	distinción de casos clásica	propiedades de los MCS
contradiction	hipótesis contradictorias	tautologías del $\leftrightarrow$
induction ... with	inducción (estructural o sobre derivaciones)	corrección, lema de verdad
subst	sustituir usando una igualdad	teorema de deducción
simp	simplificación automática	un único uso (§4.7)

Cuadro 2.1: Resumen de las tácticas empleadas en la formalización.

2.5.1. Tácticas básicas de manipulación de objetivos

intro. Si la meta es una implicación $p \rightarrow q$ o un universal $\forall x, P x$, la táctica **intro h** mueve el antecedente al contexto como nueva hipótesis **h** (o introduce un x genérico) y deja como meta el consecuente. Corresponde al “supongamos que” y al “sea x arbitrario” del lenguaje matemático. Admite varios nombres seguidos: ante la meta $\vdash p \rightarrow q \rightarrow p$, la táctica **intro hp hq** deja el contexto con **hp : p** y **hq : q** y la meta $\vdash p$. Es, con diferencia, la táctica más usada del trabajo: casi todas nuestras nociones (tautología, validez, consistencia) son cadenas de cuantificadores e implicaciones.

exact. Cierra la meta proporcionando un término que la demuestra, construido con las hipótesis disponibles. En el caso más simple el término es una hipótesis: con **hp : p** y meta $\vdash p$, basta **exact hp**. Pero puede ser cualquier término del tipo adecuado: con **h : p $\rightarrow$ q** y **hp : p** ante la meta $\vdash q$, se cierra con **exact h hp**, aplicando la implicación a la hipótesis. En el trabajo aparece a menudo en esta segunda forma, encadenando varias aplicaciones en un solo término.

apply. Razona hacia atrás: si **h : p $\rightarrow$ q** y la meta es **q**, la táctica **apply h** reduce la meta a **p** (“para ver q , por h basta ver p ”). Es la contrapartida de **exact**: en lugar de dar el término completo de una vez, se avanza dejando pendiente lo que falta. La usamos, entre otros sitios, en el caso de la implicación del lema de verdad (sección 4.7).

have. Introduce una afirmación intermedia con su propia demostración: **have h : p := término** o bien **have h : p := by** Tras ella, **h** queda disponible en el contexto como cualquier otra hipótesis. Es la táctica que estructura las demostraciones largas como sucesiones de pasos con nombre (“primero

observamos... , de donde...”). La demostración de la regla RE en la sección 4.4 es el ejemplo más evidente: siete `have` encadenados que se leen exactamente igual que la demostración en papel.

rfl. Cierra metas de la forma $a = b$ o $a \leftrightarrow b$ cuando ambos lados son definicionalmente iguales (sección 2.3): por ejemplo, la meta $\vdash 2 + 2 = 4$, que se resuelve calculando. Que un lema se demuestre con `rfl` aporta además información importante: significa que no hay contenido matemático nuevo, solo se despliegan definiciones. Lo veremos en `satisfies_neg` y en varios casos de la inducción de `cpl_eval_iff_satisfies`, en la sección 4.5.

2.5.2. Construir y descomponer estructuras lógicas

constructor. Cuando la meta es una conjunción o un bicondicional, la divide en sus componentes, generando un objetivo por cada una: ante $\vdash p \wedge q$ deja los dos objetivos $\vdash p$ y $\vdash q$, y ante $\vdash p \leftrightarrow q$, las dos implicaciones. El InfoView los muestra como objetivos separados, para delimitar la demostración de cada uno. Alternativamente puede darse el término par directamente con el constructor anónimo (sección 2.2): la demostración final del teorema de adecuación es, simplemente, `<soundness, completeness>`.

rcases, rintro, obtain. Son las tácticas duales de `constructor`: en lugar de dividir la meta, descomponen hipótesis compuestas mediante patrones. Con $h : \exists x, P x \wedge Q x$, la táctica `rcases h with <x, hP, hQ>` sustituye h por el testigo x y las dos componentes de la conjunción, $hP : P x$ y $hQ : Q x$. Los patrones pueden anidarse tanto como haga falta. Con $h : p \vee q$, en cambio, `rcases h with hp | hq` genera dos objetivos, uno por cada caso: la barra separa alternativas y los corchetes angulares agrupan componentes simultáneas. La táctica `rintro` combina `intro` con esta descomposición en un solo paso, y `obtain` hace lo propio con el resultado de aplicar un teorema: `obtain <x, hP, hQ> := teorema hp` aplica el teorema y descompone lo que devuelve. La lectura matemática es siempre la misma: “sea x el objeto que nos proporciona el teorema, con tales propiedades”. En el trabajo aparecen sobre todo al usar el lema de Lindenbaum y el lema de existencia (sección 4.7).

refine. Es un `exact` con huecos: se escribe el esqueleto del término y se dejan `?_` en las partes pendientes, que se convierten en nuevos objetivos. Ante la meta $\vdash p \wedge q$, teniendo ya $hp : p$, la táctica `refine <hp, ?_>` deja como único objetivo $\vdash q$. Resulta ideal cuando la parte creativa, elegir el testigo de un existencial, es previa a las comprobaciones rutinarias, que es exactamente la situación del lema de existencia de la sección 4.7.

2.5.3. Razonamiento clásico

by_contra. Reducción al absurdo: para demostrar p , la táctica `by_contra h` añade la hipótesis $h : \neg p$ y cambia la meta a `False`, de modo que basta llegar a una contradicción. Es un principio genuinamente clásico que equivale a la eliminación de la doble negación. Su papel en este trabajo es muy relevante: la propia demostración de completitud es una gran reducción al absurdo (sección

4.7), y en pequeño aparece cada vez que hay que pasar de una negación a una afirmación, como en la inferencia $\neg\forall x \neg P(x) \Rightarrow \exists x P(x)$, que no es demostrable intuicionistamente. Volvemos sobre esta cuestión en la sección 2.6.

by_cases. Distinción de casos sobre una proposición cualquiera: `by_cases h : p` abre dos objetivos, uno con `h : p` y otro con `h : ¬p`. Es la herramienta natural para razonar por casos sin disponer de una disyunción previa: en el trabajo la usamos para las propiedades de los conjuntos maximales consistentes (sección 4.7), del tipo “o bien Γ deduce φ , o bien no lo hace”.

contradiction. Cierra el objetivo cuando el contexto contiene una contradicción explícita: por ejemplo, `h : p` junto con `h' : ¬p`, o una hipótesis `h : False`. En tal caso la meta, sea cual sea, queda demostrada.

Relacionadas con lo anterior están dos funciones (no tácticas) que usamos para rematar razonamientos: `False.elim h`, que a partir de `h : False` demuestra cualquier cosa (*ex falso quodlibet*), y `absurd h h'`, que hace lo mismo a partir de `h : p` y `h' : ¬p`.

2.5.4. Inducción y reescritura

induction ... with. Aplica el principio de inducción del tipo inductivo correspondiente, abriendo un objetivo por constructor; la sintaxis `with | caso1 ... | caso2 ...` permite nombrar en cada caso los argumentos y las hipótesis de inducción. Un ejemplo con listas:

```

1 def longitud {α : Type} : List α → Nat
2   | []      => 0
3   | _ :: xs => longitud xs + 1
4
5 example {α : Type} (l : List α) : longitud (l ++ []) = longitud l := by
6   induction l with
7   | nil      => rfl
8   | cons x xs ih => rw [List.cons_append, longitud, longitud, ih]

```

El tipo `List α` tiene dos constructores: la lista vacía `[]` y la que antepone un elemento a otra lista, `x :: xs`. La inducción abre por tanto dos objetivos: el caso `nil`, que es la base y aquí se cierra de forma inmediata con `rfl`, y el caso `cons`, que recibe el elemento `x`, la cola `xs` y la hipótesis de inducción `ih`, que afirma la propiedad para `xs`. Lo mismo ocurre con cualquier otro tipo inductivo: hay un caso por constructor, y los constructores recursivos vienen acompañados de su hipótesis de inducción.

- *Inducción estructural sobre fórmulas* (tipo `ModalFormula`): los casos son los cuatro constructores —átomo, $\perp$, implicación y caja—, y los dos últimos traen hipótesis de inducción sobre las subfórmulas. Así se demuestran el lema puente de la corrección y el lema de verdad (secciones 4.6 y 4.7).

- *Inducción sobre derivaciones* (tipos `Provable` y `DeducibleFrom`): como una demostración formal es también un objeto inductivo, se puede razonar “por inducción sobre la demostración”, con un caso por axioma o regla. De especial relevancia para la corrección, el teorema de deducción y la monotonía de la deducibilidad.
- Con el modificador `generalizing`: cuando la propiedad que se demuestra por inducción depende de otra variable que cambia dentro de los casos, esa variable debe generalizarse antes de inducir, o las hipótesis de inducción quedarán fijadas a su valor actual y no servirán. Escribir `induction x generalizing` y lo consigue. Es el análogo formal de la precaución, habitual en papel, de enunciar la hipótesis de inducción “para todo y ”. En el lema de verdad resulta imprescindible, y allí explicamos por qué (sección 4.7).

subst. Dada una hipótesis de igualdad `heq : x = a`, elimina `x` sustituyéndolo por `a` en la meta y en las demás hipótesis, y descarta la igualdad. Es lo que en papel se hace al decir “pero entonces x es a ”, y se usa cuando una distinción de casos ha dejado como hipótesis la identidad de dos objetos que se trataban por separado.

simp. Es la principal táctica de automatización de Lean: reescribe la meta (o una hipótesis, con `simp at h`) usando una base de ecuaciones marcadas como reglas de simplificación, y cierra el objetivo si llega a algo trivial. Es muy cómoda pero tiene un precio: oculta los pasos que da, de modo que su uso indiscriminado produce demostraciones cortas pero ilegibles. Por eso en este trabajo, aparece una única vez y en un papel muy concreto: para descartar una hipótesis imposible, la pertenencia de una fórmula al conjunto vacío (sección 4.7). Todo lo demás está demostrado de forma explícita.

2.6. Mathlib, la lógica clásica y el lema de Zorn

Nuestro fichero comienza con dos importaciones:

```
1 import Mathlib.Tactic
2 import Mathlib.Order.Zorn
```

La primera carga el modo táctico extendido de Mathlib; la segunda, el lema de Zorn. De Mathlib usamos tres cosas, que repasamos a continuación.

Conjuntos

El tipo `Set  $\alpha$` representa los subconjuntos de un tipo α ; técnicamente, un conjunto es su función característica, `Set  $\alpha$  :=  $\alpha \rightarrow \text{Prop}$` , y `x  $\in$  A` es la proposición `A x`. Sobre él están definidos $\subseteq$, $\cup$, $\emptyset$, la inserción de un elemento `insert a A` (que usamos constantemente para “añadir una fórmula a un conjunto de hipótesis”) y la unión de una familia $\bigcup_0 \mathcal{C}$ (unión de todos los conjuntos que pertenecen a la colección $\mathcal{C}$). De la biblioteca tomamos lemas elementales de manipulación cuyo nombre describe su contenido: `Set.mem_insert` ($a \in \text{insert } a \ A$), `Set.mem_insert_iff` ($x \in \text{insert } a \ A \leftrightarrow x = a \vee$

$x \in A$), `Set.subset_insert`, `Set.mem_insert_of_mem`, `Set.mem_sUnion` y `Set.subset_sUnion_of_mem`. Aprender a encontrar estos lemas (por su nomenclatura sistemática, con la búsqueda del editor o con herramientas como `exact?`) ha sido parte del aprendizaje de Lean.

Lógica clásica

El núcleo de Lean es constructivo: por defecto no se dispone del tercero excluido. Mathlib, como la práctica matemática habitual, trabaja en lógica clásica, axiomatizada mediante `Classical.choice`; de ahí derivan el tercero excluido y las tácticas `by_contra` y `by_cases` que acabamos de describir. Nuestro desarrollo es clásico sin complejos, igual que el texto de referencia [3]: la propia semántica evalúa cada fórmula a verdadera o falsa, la eliminación de la doble negación es una de nuestras tautologías básicas, y la completitud usa el lema de Zorn, equivalente al axioma de elección. No es una limitación de Lean, sino una elección: existe una lógica modal intuicionista, pero no es el objeto de este trabajo.

El lema de Zorn

Del orden por inclusión entre conjuntos, Mathlib ofrece la siguiente versión del lema de Zorn, que es la que aplicaremos en el lema de Lindenbaum (sección 4.7):

```
1 theorem zorn_subset_nonempty (S : Set (Set α))
2   (H : ∀ c ⊆ S, IsChain (· ⊆ ·) c → c.Nonempty →
3     ∃ ub ∈ S, ∀ s ∈ c, s ⊆ ub)
4   (x) (hx : x ∈ S) :
5   ∃ m, x ⊆ m ∧ Maximal (· ∈ S) m
```

Leído con calma: S es una familia de conjuntos. La hipótesis H pide que toda cadena no vacía c contenida en S , (esto es, toda subfamilia totalmente ordenada por $\subseteq$, que es lo que expresa `IsChain` $(\cdot \subseteq \cdot)$ c , donde $(\cdot \subseteq \cdot)$ es la notación anónima de Lean para la relación de inclusión) tenga una cota superior dentro de S . La conclusión da, para cada $x \in S$, un elemento m de la familia con $x \subseteq m$ que es maximal en ella.

El argumento x se declara sin indicar su tipo, (x) : Lean lo deduce de la hipótesis $hx : x \in S$, que obliga a que sea un `Set α`. A su vez, la maximalidad se expresa con el predicado `Maximal (· ∈ S) m`, que significa que m cumple la propiedad de pertenecer a S y que ningún elemento que la cumpla lo contiene estrictamente. De él extraeremos las dos piezas que necesitamos con `.1` y `.2`, la pertenencia y la maximalidad propiamente dicha.

En nuestro caso S será la familia de los conjuntos consistentes de fórmulas y la cota superior de una cadena, su unión. El trabajo matemático consistirá en demostrar que esa unión sigue siendo consistente.

Capítulo 3

La lógica modal K

Este capítulo contiene la parte matemática del trabajo en su versión “de papel”: la sintaxis del lenguaje modal básico, el sistema axiomático **K**, la semántica de Kripke y los enunciados de corrección y completitud, junto con el mapa de la demostración de esta última. Todo ello sigue en lo esencial a Blackburn, de Rijke y Venema [3], con las adaptaciones, que iremos señalando, que conviene hacer pensando en la formalización del capítulo 4.

3.1. ¿Necesidad y posibilidad?

La lógica proposicional clásica trata cada enunciado como verdadero o falso, sin matices. Pero el lenguaje natural distingue *modos* de verdad: no es lo mismo “llueve” que “necesariamente llueve” o que “es posible que llueva”. La lógica modal introduce dos operadores para capturar esa distinción: $\Box\varphi$ (“ φ es necesaria”) y $\Diamond\varphi$ (“ φ es posible”). Ambos están ligados por una dualidad esperable: algo es posible exactamente cuando su negación no es necesaria, $\Diamond\varphi \equiv \neg\Box\neg\varphi$. En nuestro ejemplo de lluvia, “es posible que llueva” es lo mismo que “no es necesario que no llueva”.

Parte del atractivo de la lógica modal es que la lectura de los operadores no está fijada. Según la aplicación, un mismo par $\Box, \Diamond$ admite interpretaciones muy distintas:

- **Modalidad alética**, la lectura clásica: $\Box\varphi$ es “ φ es necesariamente cierta” y $\Diamond\varphi$, “ φ es posible”.
- **Lógica epistémica**: $\Box\varphi$ es “se sabe que φ ” y $\Diamond\varphi$, “ φ es compatible con lo que se sabe”, es decir, no puede descartarse. Los operadores suelen escribirse entonces $K\varphi$ y $\widehat{K}\varphi$.
- **Lógica temporal**: $\Box\varphi$ es “ φ será siempre cierta en el futuro” y $\Diamond\varphi$, “ φ será cierta en algún momento”.
- **Lógica deóntica**: $\Box\varphi$ es “ φ es obligatoria” y $\Diamond\varphi$, “ φ está permitida”.

En todos los casos la dualidad se mantiene, y resulta iluminadora: lo permitido es lo que no está prohibido, lo compatible con el conocimiento es lo que no se sabe falso, lo que ocurrirá alguna vez

es lo que no siempre será falso. Cada lectura sugiere, en cambio, principios distintos: por ejemplo, “lo sabido es cierto” ($\Box\varphi \rightarrow \varphi$) es razonable para el conocimiento, y también para el tiempo si se admite el instante presente, pero no para la obligación (que algo sea obligatorio no lo hace cierto). Esta multiplicidad explica que exista toda una familia de lógicas modales.

La que estudiamos aquí, la lógica **K** (por Kripke), es la base común y más débil de todas ellas: no compromete ninguna lectura y sirve de punto de partida para las demás, que se obtienen añadiendo axiomas. La lectura epistémica reaparecerá en el capítulo 5 AAA.

3.2. Sintaxis

Definición 3.1 (Lenguaje modal básico). Fijado un conjunto VP de *variables proposicionales*, las *fórmulas modales* se definen por la siguiente gramática:

$$\varphi ::= p \mid \perp \mid \varphi \rightarrow \varphi \mid \Box\varphi, \quad p \in \text{VP}.$$

Es decir, el conjunto Form de fórmulas es el menor conjunto que contiene las variables y la constante $\perp$ (“falso”) y está cerrado por la implicación y por el operador $\Box$.

Adoptamos deliberadamente un lenguaje primitivo mínimo: solo $\perp$, $\rightarrow$ y $\Box$. El resto de conectivas se introducen como abreviaturas:

$$\begin{aligned} \top &:= \perp \rightarrow \perp, & \neg\varphi &:= \varphi \rightarrow \perp, \\ \varphi \vee \psi &:= \neg\varphi \rightarrow \psi, & \varphi \wedge \psi &:= \neg(\varphi \rightarrow \neg\psi), \\ \varphi \leftrightarrow \psi &:= (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi), & \Diamond\varphi &:= \neg\Box\neg\varphi. \end{aligned}$$

Las definiciones de $\vee$ y $\wedge$ son las clásicas: $\varphi \vee \psi$ dice “si no φ , entonces ψ ”, y $\varphi \wedge \psi$ niega que φ implique la negación de ψ . La ventaja de trabajar con un lenguaje mínimo es doble. En el plano matemático, las definiciones y demostraciones por inducción estructural tienen solo cuatro casos. En el plano de la formalización, cada constructor del tipo de fórmulas es un caso en cada demostración: menos primitivas significa menos casos que demostrar, y las conectivas derivadas reciben su significado “gratis”, como comprobaremos en la sección 4.5.

Esta es la primera de las adaptaciones. Blackburn, de Rijke y Venema [3, Def. 1.9] toman como primitivos $\perp$, $\neg$, $\vee$ y $\Diamond$, y es $\Box$ el que se define, como $\Box\varphi := \neg\Diamond\neg\varphi$; la implicación es allí la abreviatura $\varphi \rightarrow \psi := \neg\varphi \vee \psi$. Nosotros invertimos la elección: $\Box$ es primitivo y $\Diamond$ es la abreviatura. Ambos lenguajes son intertraducibles (cada fórmula de uno se reescribe en el otro sin alterar su significado respecto a la semántica de la sección 3.4), y los propios autores reconocen que su preferencia por $\Diamond$ obedece a la comodidad para el trabajo algebraico de capítulos posteriores del libro. Pensando en la formalización, $\Box$ es la opción natural: su cláusula semántica es una cuantificación universal, más cómoda de manejar en un asistente de demostración que la existencial de $\Diamond$ (sección 4.5). La misma elección hace innecesario uno de los axiomas del sistema de Blackburn, como veremos en la sección 3.3.

Obsérvese también que no imponemos ninguna condición sobre el conjunto VP. Blackburn [3, p. 10] lo supone habitualmente numerable, hipótesis que interviene en su demostración del lema de Lindenbaum; nosotros podremos prescindir de ella (sección 3.5, paso 3).

Como es habitual, la inducción (y la recursión) estructural sobre fórmulas será nuestra herramienta básica: para probar que toda fórmula tiene una propiedad basta verla para p y $\perp$, y verla para $\varphi \rightarrow \psi$ y $\Box\varphi$ supuesta para las subfórmulas.

3.3. El sistema axiomático K

Un sistema axiomático determina qué fórmulas son demostrables a partir de unos axiomas mediante unas reglas de inferencia. El sistema **K** toma como punto de partida todo el cálculo proposicional clásico y le añade lo mínimo para razonar con $\Box$.

Definición 3.2 (El sistema **K**). La relación de demostrabilidad $\vdash \varphi$ (“ φ es un teorema de **K**”) es la menor relación que cumple:

(CPL) $\vdash \varphi$ para toda *tautología clásica* φ (véase más abajo);

(K) $\vdash \Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$, para cualesquiera fórmulas φ, ψ (*axioma de distribución*);

(MP) si $\vdash \varphi \rightarrow \psi$ y $\vdash \varphi$, entonces $\vdash \psi$ (*modus ponens*);

(N) si $\vdash \varphi$, entonces $\vdash \Box\varphi$ (*regla de necesidad*).

Detengámonos en (CPL). Lo habitual en los textos es dar tres esquemas de axiomas proposicionales concretos (los axiomas de Hilbert) y obtener las tautologías como teoremas. Aquí seguimos el camino, también estándar [3, p. 33], de admitir directamente todos los esquemas de tautologías; los propios autores lo describen como un “exceso axiomático” (*axiomatic overkill*) que prefieren a la alternativa de fijar unos pocos axiomas y derivar de ellos el resto, refinamiento que, a su juicio, tiene poco interés para sus propósitos, y lo mismo vale para los nuestros. Precisemos la noción: una fórmula modal es una *tautología clásica* si es verdadera bajo toda valoración booleana que asigne valores de verdad arbitrarios a sus variables y a sus subfórmulas maximales de la forma $\Box\psi$, tratadas estas como si fueran átomos opacos. La valoración no mira dentro de $\Box\psi$: en $\Box\Box p \rightarrow \Box p$, por ejemplo, los “átomos” son $\Box\Box p$ y $\Box p$, que reciben valores independientes, y por eso la fórmula no es una tautología clásica. Es fácil convencerse de que esta definición equivale a la de Blackburn, “toda instancia de una tautología proposicional”: basta sustituir cada subfórmula maximal de la forma $\Box\psi$ por una variable nueva (la misma para todas las ocurrencias de la misma subfórmula). Así, $\Box\psi \rightarrow \Box\psi$ es una tautología clásica (es una instancia de $\alpha \rightarrow \alpha$), pero $\Box p \rightarrow p$, con p una variable, no lo es: hay valoraciones que hacen verdadero el “átomo” $\Box p$ y falsa p .¹ Esta presentación evita

¹Conviene escribir $\Box p \rightarrow p$ y no $\Box\varphi \rightarrow \varphi$: si φ es ella misma una tautología, como $\top$, entonces $\Box\varphi \rightarrow \varphi$ sí es una tautología clásica, porque ninguna valoración hace falsa φ .

desarrollar dentro de **K** toda la maquinaria del cálculo proposicional, que no es el objeto del trabajo, y tiene una traducción muy limpia a Lean, donde “ser tautología” se puede definir semánticamente (sección 4.3). El precio es que (CPL) es un esquema infinito, algo que a un sistema formal como Lean no le supone ningún problema.

El sistema de la definición 3.2 no es literalmente el **K** de Blackburn [3, Def. 1.39], y conviene dejar claro en qué difieren y por qué tienen los mismos teoremas. Allí **K** cuenta con un tercer axioma, (Dual) $\Diamond p \leftrightarrow \neg \Box \neg p$, y con una tercera regla, la *sustitución uniforme*: de $\vdash \varphi$ se pasa a $\vdash \varphi^\sigma$, donde φ^σ resulta de sustituir uniformemente las variables de φ por fórmulas cualesquiera. Además, los axiomas (K) y (Dual) se enuncian con variables concretas p, q , y no como esquemas.

- (Dual) sobra en nuestro lenguaje. Como aquí $\Diamond \varphi$ es por definición $\neg \Box \neg \varphi$, el axioma se desplegaría a $\neg \Box \neg \varphi \leftrightarrow \neg \Box \neg \varphi$, una instancia de $\alpha \leftrightarrow \alpha$, que ya está en (CPL). Los propios autores lo advierten [3, p. 34]: (Dual) solo hace falta porque su primitivo es $\Diamond$, y de haber tomado $\Box$ como primitivo no habría sido necesario.
- La *sustitución uniforme* sobra porque nuestros axiomas son esquemas: (K) se postula de golpe para todas las fórmulas φ, ψ , y el conjunto de las tautologías es cerrado bajo sustitución por su propia definición. Con ello, la cerradura de los teoremas bajo sustitución deja de ser una regla y pasa a ser un teorema, que se demuestra por inducción sobre la derivación: las instancias de tautologías son tautologías, las instancias de (K) son instancias de (K), y las reglas (MP) y (N) conmutan con la sustitución.

Así pues, formulados en nuestro lenguaje, el sistema de Blackburn y el de la definición 3.2 demuestran exactamente las mismas fórmulas: cada uno contiene los axiomas del otro y es cerrado bajo sus reglas. Dicho de otro modo, el conjunto de nuestros teoremas contiene las tautologías, (K) y (Dual), y es cerrado bajo (MP), (N) y sustitución uniforme, es decir, es una *lógica modal normal* en el sentido de [3, Def. 1.42]; y, al estar generado por lo imprescindible, es la menor de todas ellas, que es justamente como se define **K** en esa definición. De hecho, nuestra definición “como la menor relación que cumple...” está más cerca de esa caracterización conjuntista que de la definición 1.39 (demostraciones como sucesiones finitas de fórmulas); la equivalencia entre ambos puntos de vista es el ejercicio 1.6.6 de [3], y la versión “menor relación” es precisamente la que produce una definición inductiva en Lean.

Los axiomas (K) y la regla (N) capturan el comportamiento mínimo de $\Box$: la necesidad distribuye sobre la implicación, y los teoremas son necesarios. Nótese lo que la regla (N) no dice: no afirma $\vdash \varphi \rightarrow \Box \varphi$ (las verdades contingentes no son necesarias), sino solo que las fórmulas demostrables pueden “encajonarse”. Blackburn dedica a este punto una advertencia expresa [3, Obs. 1.41]: quien venga de la deducción natural puede sentirse tentado de suponer p , aplicar (N) para obtener $\Box p$ y “descargar” la hipótesis concluyendo $p \rightarrow \Box p$, que no es válida. El error está en aplicar (N) a

una hipótesis: la regla solo se aplica a teoremas. Volveremos sobre ello al definir la deducción desde hipótesis (sección 3.5).

De estos ingredientes se derivan reglas útiles. Destacamos las dos que formalizaremos, con sus demostraciones en papel, que después podrán compararse línea a línea con el código:

Proposición 3.3 (Regla de monotonía, RM). *Si $\vdash \varphi \rightarrow \psi$, entonces $\vdash \Box\varphi \rightarrow \Box\psi$.*

Demostración. Por (N), de $\vdash \varphi \rightarrow \psi$ obtenemos $\vdash \Box(\varphi \rightarrow \psi)$. El axioma (K) da $\vdash \Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$, y (MP) concluye. $\square$

Proposición 3.4 (Regla de equivalencia, RE). *Si $\vdash \varphi \leftrightarrow \psi$, entonces $\vdash \Box\varphi \leftrightarrow \Box\psi$.*

Demostración. De $\vdash \varphi \leftrightarrow \psi$ se extraen, usando las tautologías $(\varphi \leftrightarrow \psi) \rightarrow (\varphi \rightarrow \psi)$ y $(\varphi \leftrightarrow \psi) \rightarrow (\psi \rightarrow \varphi)$ junto con (MP), las dos implicaciones $\vdash \varphi \rightarrow \psi$ y $\vdash \psi \rightarrow \varphi$. La regla RM da entonces $\vdash \Box\varphi \rightarrow \Box\psi$ y $\vdash \Box\psi \rightarrow \Box\varphi$, y la tautología $(\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \alpha) \rightarrow (\alpha \leftrightarrow \beta))$, con dos aplicaciones de (MP), las reúne en la equivalencia buscada. $\square$

Obsérvese el patrón de estas demostraciones: los únicos movimientos permitidos son citar un axioma y aplicar una regla. Es un estilo rígido pero es exactamente esta rigidez la que lo hace formalizable de manera directa.

3.4. Semántica de Kripke

La semántica estándar de la lógica modal, debida a Kripke [6], se apoya en una idea muy visual: la de *mundos posibles* conectados por una relación de accesibilidad. Una fórmula no es verdadera o falsa en abstracto, sino en un mundo; y los operadores modales cuantifican sobre los mundos accesibles desde el actual.

Definición 3.5 (Marcos y modelos). Un *marco de Kripke* es un par $F = (W, R)$ formado por un conjunto no vacío² W de mundos y una relación binaria $R \subseteq W \times W$, la *relación de accesibilidad*. Un *modelo de Kripke* sobre VP es un par $M = (F, V)$ donde F es un marco y $V: W \rightarrow \mathcal{P}(\text{VP})$ es una *valoración* que indica qué variables son verdaderas en cada mundo.

Un marco es, sencillamente, un grafo dirigido; escribimos $R w v$ o $w R v$ para “desde w es accesible v ”. En la valoración hay otra pequeña adaptación respecto a [3, Def. 1.19], donde $V: \text{VP} \rightarrow \mathcal{P}(W)$ asigna a cada variable el conjunto de mundos en que es verdadera, de modo que la cláusula atómica de la definición siguiente se lee allí $w \in V(p)$. Nosotros la escribimos al revés, $V: W \rightarrow \mathcal{P}(\text{VP})$, y leeremos $p \in V(w)$. La información es la misma (en ambos casos, un subconjunto de $W \times \text{VP}$), pero nuestra dirección es la que usaremos en la figura 3.1, en el modelo canónico de la sección 3.5 (donde la valoración de un mundo Γ es el conjunto de variables que pertenecen a Γ) y en el código, donde la valoración recibe primero el mundo y después la variable.

²En la formalización el tipo de mundos podrá ser vacío; véase la observación 4.1 en la sección 4.5.

Definición 3.6 (Satisfacción). Dado un modelo M y un mundo w , la relación $M, w \models \varphi$ (“ φ es verdadera en el mundo w de M ”) se define por recursión sobre φ :

$$\begin{array}{ll} M, w \models p & \iff p \in V(w), \\ M, w \models \perp & \text{no se da nunca,} \\ M, w \models \varphi \rightarrow \psi & \iff \text{si } M, w \models \varphi \text{ entonces } M, w \models \psi, \\ M, w \models \Box \varphi & \iff \text{para todo } v \text{ con } R w v, M, v \models \varphi. \end{array}$$

La cláusula interesante es la última: $\Box \varphi$ es verdadera en w cuando φ lo es en todos los mundos accesibles desde w . Desplegando las abreviaturas se comprueba que las conectivas derivadas reciben el significado esperado; en particular,

$$M, w \models \Diamond \varphi \iff \text{existe } v \text{ con } R w v \text{ y } M, v \models \varphi :$$

la posibilidad es la cuantificación existencial sobre los mundos accesibles.

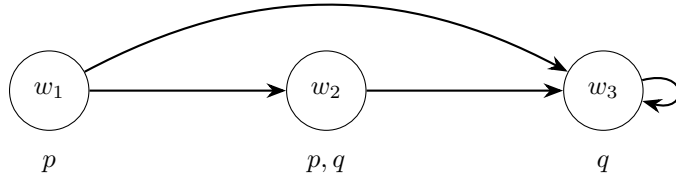


Figura 3.1: Un modelo de Kripke con tres mundos; bajo cada mundo se listan las variables que su valoración hace verdaderas. Se cumple, por ejemplo, $M, w_1 \models \Diamond p$ (hay un mundo accesible, w_2 , donde vale p); en cambio $M, w_1 \not\models \Box p$, porque p falla en el mundo accesible w_3 . También $M, w_2 \models \Box q$: el único mundo accesible desde w_2 es w_3 , y allí vale q . En w_3 , cuyo único sucesor es él mismo, valen tanto $\Box q$ como $\Diamond q$; sin el bucle, w_3 no tendría sucesores y $\Box q$ seguiría siendo verdadera, pero vacuamente, mientras que $\Diamond q$ pasaría a ser falsa (véase la observación siguiente).

Observación 3.7. En un mundo sin sucesores, $\Box \varphi$ es verdadera vacuamente para cualquier φ (¡incluida $\perp$!), pues la condición “en todo mundo accesible...” no tiene nada que comprobar. Dualmente, $\Diamond \varphi$ es falsa para toda φ . Este fenómeno, que puede resultar chocante, es inofensivo en **K** y volverá a aparecer en la formalización: el modelo con el que probaremos la consistencia del sistema es, precisamente, un único mundo sin sucesores.

Definición 3.8 (Validez). Una fórmula φ es *válida en un modelo* M , escrito $M \models \varphi$, si $M, w \models \varphi$ para todo mundo w de M ; es *válida en un marco* F si es válida en todo modelo sobre F ; y es *válida* (sin más), escrito $\models \varphi$, si es válida en todo modelo de Kripke.

Una advertencia terminológica. Blackburn [3, Def. 1.21] llama a la primera noción *verdad global* (o *universal*) en el modelo, y reserva la palabra “válida” para los marcos y las clases de marcos [3, Def. 1.28]: solo al cuantificar sobre todas las valoraciones se abstrae de la información contingente que aporta V y se llega a una noción propiamente lógica. Hemos preferido un único nombre para los tres niveles, que es también el que usa el código (`ValidInModel`, `ValidInFrame` y `Valid`), pero conviene tener presente la distinción al consultar el libro.

La noción central para nosotros es la última, porque es la contrapartida semántica de la demostrabilidad en **K**. La validez en un marco, que no depende de la valoración, es la puerta de entrada a las lógicas modales más fuertes: por ejemplo, $\Box p \rightarrow p$ (con p una variable) es válida en un marco exactamente cuando este es reflexivo, y añadirla como axioma da la lógica **T**.³ En este trabajo la definimos por completitud del planteamiento, pero no la explotamos.

Dos ejemplos orientan sobre qué es válido y qué no.

La fórmula $\Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$ (el axioma K) es válida: si en todo mundo accesible valen $\varphi \rightarrow \psi$ y φ , en todo mundo accesible vale ψ ; es un “modus ponens mundo a mundo”.

En cambio, $\Box p \rightarrow p$ no es válida: en un modelo con dos mundos $w R v$, sin más flechas, con p verdadera solo en v , se tiene $M, w \models \Box p$ pero no $M, w \models p$. Como **K** demostrará exactamente las fórmulas válidas, este contraejemplo muestra también que $\Box p \rightarrow p$ no es un teorema de **K**.

3.5. Corrección y completitud: el mapa de la demostración

Tenemos dos nociones de “verdad lógica”: la sintáctica ($\vdash \varphi$) y la semántica ($\models \varphi$). Los dos teoremas centrales de este trabajo afirman que coinciden.

Teorema 3.9 (Corrección). *Si $\vdash \varphi$, entonces $\models \varphi$.*

Teorema 3.10 (Completitud). *Si $\models \varphi$, entonces $\vdash \varphi$.*

Son los enunciados que formalizaremos (**soundness** y **completeness** en el capítulo 4). Conviene saber que Blackburn demuestra algo más fuerte [3, Teorema 4.23], la llamada completitud *fuerte*: para todo conjunto de fórmulas Σ , si φ es consecuencia semántica de Σ (es decir, φ es verdadera en todo mundo de todo modelo en el que sean verdaderas todas las fórmulas de Σ), entonces φ se deduce de Σ en **K**. El teorema 3.10 es el caso $\Sigma = \emptyset$. En este trabajo nos limitamos a la versión débil; la construcción que describimos a continuación es, en cualquier caso, la misma que da la fuerte.

La corrección es la dirección fácil, y su demostración es una inducción sobre la derivación de $\vdash \varphi$: basta comprobar que los axiomas (las tautologías y el axioma K) son válidos y que las reglas (MP y N) preservan la validez. El único punto delicado es el caso (CPL): hay que conectar la noción booleana de tautología (donde las subfórmulas $\Box\psi$ son átomos opacos) con la satisfacción en un mundo concreto. La clave es que, fijado un mundo w , podemos “leer” en él una valoración booleana —la que asigna a cada átomo y a cada $\Box\psi$ su valor de verdad en w — y comprobar, por inducción estructural, que evaluar clásicamente con esa valoración coincide con la satisfacción en w . Este lema puente (`cpl_eval_iff_satisfies` en la formalización) hace todo el trabajo.

Una consecuencia inmediata de la corrección es la *consistencia* del sistema: $\not\vdash \perp$, pues $\perp$ falla en cualquier mundo de cualquier modelo (y existe al menos un modelo con un mundo).

³De nuevo importa escribir $\Box p \rightarrow p$ y no $\Box\varphi \rightarrow \varphi$: la instancia $\Box\top \rightarrow \top$ es válida en todo marco. Lo que caracteriza a los marcos reflexivos es la validez de la fórmula con una variable o, equivalentemente, la de todas las instancias del esquema. Estas correspondencias entre fórmulas y propiedades de los marcos son el tema del capítulo 3 de [3].

La completitud es sustancialmente más profunda: hay que fabricar, para cada fórmula no demostrable, un modelo que la refute. La estrategia clásica, que seguimos de [3], construye de una vez un *modelo canónico* que refuta simultáneamente todas las fórmulas no demostrables. El plan, cuyos pasos formalizaremos uno a uno en la sección 4.7, es el siguiente:

1. **Deducibilidad desde hipótesis.** Se generaliza $\vdash \varphi$ a una relación $\Gamma \vdash \varphi$ (“ φ se deduce del conjunto de hipótesis Γ ”), con su *teorema de deducción*: si $\Gamma \cup \{\varphi\} \vdash \psi$ entonces $\Gamma \vdash \varphi \rightarrow \psi$. Aquí nos separamos de [3, Def. 4.4], que define $\Gamma \vdash \varphi$ mediante conjunciones finitas de premisas: o bien $\vdash \varphi$ directamente, o bien $\vdash (\psi_1 \wedge \cdots \wedge \psi_n) \rightarrow \varphi$ para ciertas $\psi_1, \dots, \psi_n \in \Gamma$; nosotros la definimos inductivamente, con tres reglas (usar un teorema, usar una hipótesis, modus ponens). Ambas definiciones son equivalentes, pero la inductiva es mucho más cómoda de formalizar: evita manipular listas de fórmulas y conjunciones anidadas, y regala un principio de inducción sobre deducciones. Es importante notar que la regla de necesidad no se incluye entre las reglas de deducción desde hipótesis: de $\Gamma \vdash \varphi$ no debe seguirse $\Gamma \vdash \Box \varphi$ (mis hipótesis pueden ser verdades contingentes). Es exactamente la trampa de la observación 1.41 de [3] que comentamos en la sección 3.3. En la definición por conjunciones finitas el peligro no existe, porque (N) solo se aplica al teorema $\vdash (\psi_1 \wedge \cdots \wedge \psi_n) \rightarrow \varphi$; en la inductiva hay que excluirla a propósito.
2. **Consistencia y conjuntos maximales.** Un conjunto Γ es *consistente* si $\Gamma \not\vdash \perp$, y *maximal consistente* (MCS) si además ninguna extensión propia suya es consistente [3, Def. 4.15]. Los MCS se comportan como “descripciones completas de un mundo” [3, Prop. 4.16]: están deductivamente cerrados, para cada φ contienen exactamente una de φ , $\neg \varphi$, y contienen $\varphi \rightarrow \psi$ si y solo si contener φ los obliga a contener ψ .
3. **Lema de Lindenbaum.** Todo conjunto consistente se extiende a un MCS. La prueba de [3, Lema 4.17] enumera las fórmulas $(\varphi_0, \varphi_1, \dots)$ y las va añadiendo una a una cuando no rompen la consistencia; esto exige que el lenguaje sea numerable. Nosotros seguimos el camino alternativo del lema de Zorn: la familia de los conjuntos consistentes, ordenada por inclusión, tiene la propiedad de que toda cadena está acotada (por su unión, que es consistente porque toda deducción usa finitas hipótesis), luego posee elementos maximales sobre cualquier consistente dado. Se elimina así la hipótesis de numerabilidad, y la formalización se beneficia del lema de Zorn ya disponible en Mathlib.
4. **El modelo canónico.** Sus mundos son todos los MCS; la accesibilidad declara $\Gamma R \Delta$ cuando Δ cumple todas las “obligaciones” modales de Γ (si $\Box \psi \in \Gamma$ entonces $\psi \in \Delta$); y la valoración hace verdadera p en Γ exactamente cuando $p \in \Gamma$. Blackburn [3, Def. 4.18] define la accesibilidad canónica con $\Diamond$ (si $\psi \in \Delta$ entonces $\Diamond \psi \in \Gamma$) y demuestra en su lema 4.19 que equivale a la formulación con $\Box$ que adoptamos aquí, la natural cuando $\Box$ es el primitivo.

5. **Lema de existencia.** Si $\Box\varphi \notin \Gamma$ (con Γ un MCS), existe un MCS Δ accesible desde Γ con $\varphi \notin \Delta$. Es la forma dual del lema 4.20 de [3] (allí: si $\Diamond\varphi \in \Gamma$, existe Δ accesible con $\varphi \in \Delta$), y la demostración es la misma. Se construye aplicando Lindenbaum al conjunto $\{\psi : \Box\psi \in \Gamma\} \cup \{\neg\varphi\}$, cuya consistencia hay que comprobar; el ingrediente clave es que si $\{\psi : \Box\psi \in \Gamma\} \vdash \varphi$ entonces $\Gamma \vdash \Box\varphi$ (todas las reglas de la deducción se pueden “encajonar” gracias al axioma K y a la necesidad).
6. **Lema de verdad.** En el modelo canónico, verdad y pertenencia coinciden: $M^c, \Gamma \models \varphi \iff \varphi \in \Gamma$, para toda fórmula φ [3, Lema 4.21]. Se demuestra por inducción estructural; los casos proposicionales salen de las propiedades de los MCS, y el caso $\Box\varphi$ combina la definición de la accesibilidad canónica (una dirección) con el lema de existencia (la otra).
7. **Conclusión, por contraposición.** Si $\not\models \varphi$, entonces $\{\neg\varphi\}$ es consistente, Lindenbaum lo extiende a un MCS Γ , y el lema de verdad da $M^c, \Gamma \not\models \varphi$: la fórmula φ no es válida. Es el argumento de los teoremas 4.22 y 4.23 de [3], reducido al caso de una sola fórmula.

AAA

Capítulo 4

La formalización en Lean

AAA

4.1. Visión general y decisiones de diseño

AAA

4.2. La sintaxis

AAA

4.3. El esquema de tautologías

AAA

4.4. El sistema axiomático

AAA

4.5. La semántica de Kripke

AAA

Observación 4.1. No exigimos que `World` sea no vacío, a diferencia de la definición 3.5. Sobre el modelo (único) con tipo de mundos vacío toda fórmula es válida vacuamente, lo cual es inofensivo: la validez cuantifica sobre *todos* los modelos, y sobra con que exista alguno no vacío para que $\perp$ no sea válida. A cambio, las definiciones quedan más limpias (sin arrastrar una prueba de no vacuidad) y los teoremas, marginalmente más generales.

4.6. El teorema de corrección

AAA

4.7. El teorema de completitud

AAA

4.7.1. Deducción a partir de hipótesis

AAA

4.7.2. Consistencia y conjuntos maximales consistentes

AAA

4.7.3. El lema de Lindenbaum, vía Zorn

AAA

4.7.4. El modelo canónico y el lema de encajonado

AAA

4.7.5. Los lemas de existencia y de verdad

AAA

4.7.6. Completitud y adecuación

AAA

Capítulo 5

Hacia la lógica epistémica: el problema de los niños embarrados

AAA

Capítulo 6

Conclusiones

AAA

Bibliografía

- [1] J. Avigad, L. de Moura, S. Kong, and S. Ullrich. Theorem proving in Lean 4. https://lean-lang.org/theorem_proving_in_lean4/, 2024. Consultado en 2026.
- [2] J. Avigad and P. Massot. Mathematics in Lean. https://leanprover-community.github.io/mathematics_in_lean/, 2024. Consultado en 2026.
- [3] P. Blackburn, M. de Rijke, and Y. Venema. *Modal Logic*, volume 53 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2001.
- [4] K. Buzzard et al. The natural number game (Lean 4). <https://adam.math.hhu.de/>, 2023. Consultado en 2026.
- [5] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*. Springer, 2021.
- [6] S. Kripke. Semantical analysis of modal logic I: Normal modal propositional calculi. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik*, 9:67–96, 1963.
- [7] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, 2020.